PATRICK
CONNOLLY

[FOUNDER TOOLKIT] PRODUCT DESIGN TOOLKIT

Designing / what you build.

The full product-design toolkit, from **understanding the user** to **writing the spec** — sequenced by altitude, so each tool earns the one below it. The companion to the Pilot Playbook.

June 2026

Developed for founders I mentor at the International Space University and ISU Incubator

HOW TO READ THIS TOOLKIT

Four altitudes / one descent.

Read top to bottom. Each altitude assumes the one above it is done: you can't map a journey before you know the segment, and you can't prioritize features before you have use cases. Higher is fuzzier and about **people**; lower is sharper and about **build**.

ALTITUDE 1

Who & why

Understand the user

The human, not the feature

Jobs-to-be-Done · **user segments & prioritization** · user profiles / personas · customer journey map

ALTITUDE 2

What to solve

Frame the problem

Converge on the bet

Use cases · **the Google Design Sprint** as the connective method

ALTITUDE 3

How it behaves

Design the solution

Paths through the product

Workflows · **user flows** — primary, alternate, contingency, emergency

ALTITUDE 4

What to build

Specify for build

Written down, not over-written

Feature prioritization · functional requirements · **performance / non-functional requirements**

ALTITUDE 1 · WHO & WHY

Understand / the user.

Before a single feature, get honest about who you serve and what they're really trying to get done. Everything downstream inherits the errors you make here.

Jobs-to-be-Done | Segments & prioritization | Personas | Journey map

1

TOOL 1.1 · JOBS-TO-BE-DONE

Jobs-to-be-Done

People don't buy products — they hire them to make progress in a situation. Find the job, and you stop guessing at features.

WHAT IT IS

A **job** is the progress a user is trying to make in a given circumstance, including its functional, emotional, and social dimensions. The classic frame: **"When ___ , I want to ___ , so I can ___ ."** The job is stable even when products change; features are just one way to get it done. Designing for the job — not the demographic — is what keeps you from building a faster horse.

THE QUESTIONS IT ANSWERS

- *What progress is the user trying to make, and in what **situation**?*
- *What do they **hire** today to do it — including spreadsheets, workarounds, or nothing?*
- *What would make them **fire** the current solution?*
- *What does "done" feel like for them — functionally and emotionally?*

WHAT "GOOD" LOOKS LIKE

- ✓ A job statement in the When/I-want/so-I-can form
- ✓ The job is solution-agnostic (no feature names in it)
- ✓ You can name the current workaround being fired
- ✓ It came from real interviews, not your imagination

FAILURE MODE

Writing a feature in job's clothing. "When I open the app, I want a dashboard" is a feature with a costume on. A real job survives even if your product never exists — keep asking "so that I can…?" until you hit a motive, not a UI.

THE SHAPE OF A JOB

When (situation) + **I want to (motivation)** → **so I can (outcome)**

AEGIS EO

Aegis's job statement, pulled straight from **52** conversations:

"When a property claim comes in, I want to verify what actually happened at that address from imagery I already pay for, so I can settle the claim faster and with less dispute."

Note what's **absent**: no mention of dashboards, satellites, or AI. The job was about **settling claims faster** — which is why "time-to-claim" became the metric everything else pointed at.

TOOL 1.2 · SEGMENTS & PRIORITIZATION

User segments / & prioritization

Not all users are your users yet. Group them by shared job and context, then pick the one to win first — the same beachhead logic, one level down.

WHAT IT IS

Segmentation splits the market into groups that share a job, context, and buying behaviour. **Prioritization** ranks those segments so you serve one deeply rather than all shallowly. Good segments are defined by **behaviour and job**, not just firmographics or demographics — “underwriters who already pay for imagery but can't use it” is a segment; “large companies” is not.

THE QUESTIONS IT ANSWERS

- *What distinct **jobs or contexts** separate one group from another?*
- *Which segment feels the pain **most acutely** and can act on it?*
- *Which is **reachable**, and representative of the next ten?*
- *Which do you explicitly **defer** — and why?*

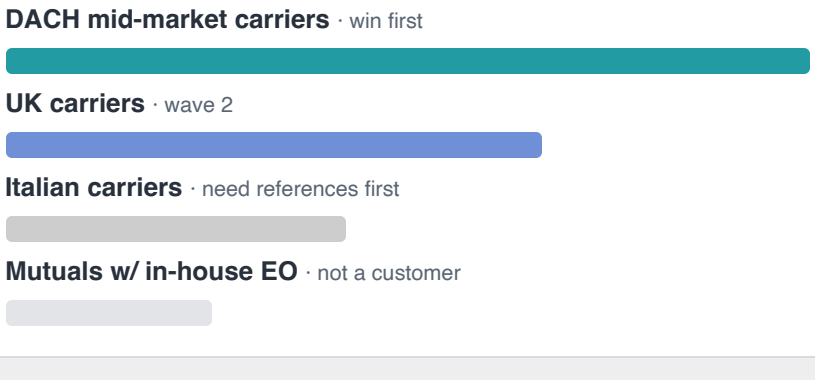
WHAT "GOOD" LOOKS LIKE

- ✓ Segments defined by behaviour/job, not size alone
- ✓ One primary segment chosen on written criteria
- ✓ Deferred segments named, not forgotten
- ✓ The primary segment matches your pilot customer

FAILURE MODE

Boiling the ocean. Treating “insurers” as one segment hides the fact that a Munich mutual with its own EO team and a mid-market DACH carrier have opposite jobs. Under-segmenting makes the product vague; over-segmenting makes it un-buildable. Aim for the few cuts that change what you'd build.

AEGIS SEGMENT PRIORITY



AEGIS EO

Aegis ranked carrier segments by acuteness and reachability, then committed to **one**:

DACH mid-market carriers won first — acute pain, reachable, and representative of the next ten. UK was wave 2; Italian carriers **needed references** before they'd engage; mutuals with in-house EO teams were named explicitly as **not a customer**.

TOOL 1.3 · USER PROFILES

User profiles / & personas

A persona turns a segment into a person you can design for and argue about. Used well it's a decision tool; used badly it's a stock photo with a fake name.

— WHAT IT IS

A **persona** is a concrete, evidence-based portrait of a representative user: their job, context, goals, constraints, and the words they actually use. In B2B you usually need a small **cast**, not one persona — because the person who feels the pain, the person who pays, and the person who can block you are rarely the same (see the Pilot Playbook's DMU). A persona is only as good as the interviews behind it.

— THE QUESTIONS IT ANSWERS

- *Who exactly are we designing for — in their own **words**?*
- *What are their **goals**, and what's in their way?*
- *Where do the **payer, user, and blocker** differ?*
- *What would make this person a **champion** internally?*

— WHAT "GOOD" LOOKS LIKE

- ✓ Grounded in real quotes, not invented traits
- ✓ A small cast covering user / buyer / blocker
- ✓ Goals and constraints, not just demographics
- ✓ Specific enough to settle a design argument

— FAILURE MODE

The demographic decoy. “34, urban, likes innovation” tells you nothing about what to build. If swapping the photo and name wouldn't change a single product decision, the persona is decoration. Anchor every trait to something you heard.

UD **The Underwriting-Data Lead**
CHAMPION · DACH carrier

> *“We pay for the imagery. We just can't get it into the claims process.”*

GOAL	Settle claims faster, with fewer disputes
BLOCKED BY	No integration; PII rules; Group IT owns budget
WINS IF	Time-to-claim drops on a real workflow

AEGIS EO

Aegis's primary persona doubled as its pilot **champion** — the person who felt the pain daily and could sell upward.

TOOL 1.4 · CUSTOMER JOURNEY MAP

Customer journey map

Lay out the user's experience as a sequence of stages — actions, thoughts, and emotions — to find where the pain spikes and where you can intervene.

— WHAT IT IS

A **journey map** visualizes the end-to-end experience of getting the job done, stage by stage, capturing what the user **does**, **thinks**, and **feels** at each step. Its power is locating the **moments that matter** — the emotional low points and drop-offs where a product can create the most value. It connects the abstract job to concrete touchpoints you can design.

— THE QUESTIONS IT ANSWERS

- *What are the **stages** of getting this job done today, start to finish?*
- *Where does **emotion dip** — frustration, confusion, risk?*
- *Which step is the **highest-value** place to intervene?*
- *Where do users **drop out** or fall back to workarounds?*

— WHAT "GOOD" LOOKS LIKE

- ✓ Stages reflect the user's reality, not your funnel
- ✓ Emotional highs and lows are marked
- ✓ One clear “moment that matters” identified
- ✓ It points to a specific place your pilot acts

— FAILURE MODE

Mapping your product instead of their life. If the stages are “sign up → onboard → upgrade,” you've drawn your funnel, not their journey. The map should make sense even to someone who's never heard of you — the product appears only where it helps.

1 · CLAIM IN	2 · PULL DATA	3 · CORRELATE	4 · DECIDE
Receive	**Fetch imagery**	**Match to claim**	**Settle**
Property claim arrives	Already subscribed	Manual, slow, or skipped	Delayed, disputed
😐	🙂	😠	😕

AEGIS EO

Mapping the claims journey showed Aegis the pain wasn't in getting imagery — it was one step later, at **correlating it to the claim**. That's the moment that mattered, and exactly where the pilot intervened.

ALTITUDE 2 · WHAT TO SOLVE

Frame / the problem.

With the user understood, converge: turn messy insight into specific use cases and a testable design direction. This is where discovery becomes a bet.

Use cases

The Google Design Sprint

2

TOOL 2.1 · USE CASES

Use cases

A use case is a specific interaction in which the user achieves a goal. It's the bridge from “who they are” to “what the thing must do.”

— WHAT IT IS

A **use case** describes how an **actor** interacts with the system to achieve a **goal**, including the trigger, the main success path, and what happens when things vary. It's more concrete than a job and more human than a requirement — the connective tissue between altitude 1 and altitude 4. Each use case should name an actor, a goal, a trigger, and an outcome.

— THE QUESTIONS IT ANSWERS

- *Who is the **actor**, and what **goal** do they achieve?*
- *What **triggers** the interaction?*
- *What's the **main success path**, in plain steps?*
- *Which use case is the **core** one your pilot must nail?*

— WHAT "GOOD" LOOKS LIKE

- ✓ Actor + goal + trigger + outcome all named
- ✓ Written as steps, not features
- ✓ A clear primary use case identified
- ✓ Edge variations noted but not yet detailed

— FAILURE MODE

Use-case sprawl. Listing forty use cases before building anything is procrastination dressed as rigor. For a pilot you need **one** primary use case nailed cold; the rest are a backlog, not a blocker.

UC-01 · Verify a property claim	
ACTOR	Underwriting-data lead at a DACH carrier
TRIGGER	A new property claim is filed
GOAL	Confirm what happened at the address from existing imagery
OUTCOME	Claim settled faster; decision evidenced

AEGIS EO

Aegis's primary use case was deliberately singular — the smallest interaction that proved the bet.

TOOL 2.2 · THE GOOGLE DESIGN SPRINT

The Google / Design Sprint

A five-day method to go from a hard problem to a tested prototype — without building the real thing. The fastest honest way to de-risk a design direction.

WHAT IT IS

The **Design Sprint** compresses months of debate into one structured week: **Map** the problem, **Sketch** solutions, **Decide** on one, **Prototype** a realistic façade, and **Test** it with five real users. The output isn't a product — it's **evidence** about whether a direction is worth building. It pairs naturally with the Pilot Playbook's "derisk without over-investing": prototype the experience, fake the back end.

THE QUESTIONS IT ANSWERS

- *What's the **one big question** the week must answer?*
- *Which solution sketch do we **commit** to prototype?*
- *What's the **thinnest prototype** that feels real to a user?*
- *What did **five real users** actually do with it?*

WHAT "GOOD" LOOKS LIKE

- ✓ A single sprint question, agreed up front
- ✓ A decided concept, not a committee compromise
- ✓ A prototype faked convincingly, built in a day
- ✓ Tested with real target users, not teammates

FAILURE MODE

Sprinting without users. A sprint that ends in a prototype no real customer touches is just an offsite. The test on day five is the entire point — skip it and you've spent a week reinforcing your own opinions.

THE FIVE DAYS

MON	TUE	WED	THU	FRI
Map	Sketch	Decide	Prototype	Test

AEGIS EO

Before committing engineering, Aegis could run the workflow concept past Munich Re's team as a **prototype** — the same logic as their concierge pilot: **make the experience real, fake the automation**, and watch what users do.

ALTITUDE 3 · HOW IT BEHAVES

Design / the solution.

Now define behaviour: the workflows the product supports and the paths users take through it — not just the happy one, but the alternates, the contingencies, and the emergencies.

Workflows | Primary flow | Alternate & contingency | Emergency

3

TOOL 3.1 · WORKFLOWS

Workflows

A workflow is the sequence of steps — across people and systems — that gets the job done. Design the workflow before the screens.

WHAT IT IS

A **workflow** describes the end-to-end process of accomplishing a task, including the handoffs between **people, systems, and steps**. It's broader than a single user's flow because it includes everyone and everything involved. In B2B especially, the product has to slot into an existing workflow — so map the current-state workflow first, then design the future-state one your product enables.

THE QUESTIONS IT ANSWERS

- *What's the **current** workflow, including manual workarounds?*
- *Where are the **handoffs** between people and systems?*
- *What does the **future** workflow look like with you in it?*
- *Which single step does your product actually **change**?*

WHAT "GOOD" LOOKS LIKE

- ✓ Current state mapped before future state
- ✓ Handoffs and systems shown, not just user steps
- ✓ The product's insertion point is explicit
- ✓ It's a process, not a screen flow

FAILURE MODE

Designing the screen before the system. Jumping to UI before mapping the workflow produces beautiful interfaces that don't fit how work actually moves. The integration point — where your product meets their existing systems — is usually where the real design problem lives.

FUTURE-STATE WORKFLOW

Claim filed → **Aegis pipes imagery in** → **Analyst reviews in-system** → **Settle**

AEGIS EO

Aegis's whole thesis was a **workflow** insight: the imagery existed, the subscription existed — the broken handoff was getting it **into the claims system**. The product was the integration step, not the data.

TOOL 3.2 · USER FLOWS

User flows / four kinds of path

A user flow is the path a single user takes to complete a task. Mature products design four: the primary path, and what happens when reality intervenes.

WHAT IT IS

A **user flow** traces one user's route to a goal, screen by screen or step by step. Designing only the **primary** (happy) path is what separates a demo from a product. The four types: **Primary** — everything works. **Alternate** — a valid different route to the same goal. **Contingency** — something fails and the user recovers. **Emergency** — a rare, high-stakes path where the cost of getting it wrong is severe.

THE QUESTIONS IT ANSWERS

- *What's the **primary** path when everything works?*
- *What **alternate** routes reach the same goal?*
- *How does the user **recover** when a step fails (contingency)?*
- *What's the **emergency** path, and is it safe by design?*

WHAT "GOOD" LOOKS LIKE

- ✓ Primary path designed end-to-end first
- ✓ At least the likely failure points have a contingency
- ✓ Emergency paths identified for high-stakes actions
- ✓ You haven't over-built rare paths for a pilot

FAILURE MODE

Shipping only the happy path. The demo works; the product breaks the first time a user does something unexpected. But the opposite trap is real too: fully specifying emergency flows for a four-week pilot is wasted motion. Design primary now, sketch contingencies, defer the rare ones.

THE FOUR FLOWS

PRIMARY	Open claim → Imagery shown → Settle
ALTERNATE	Bulk queue → Batch review
CONTINGENCY	No imagery → Flag for manual
EMERGENCY	Bad match risk → Block auto-settle

AEGIS EO

For the pilot, Aegis nailed the **primary** flow and only sketched the rest. The **contingency** that mattered most: what happens when imagery for an address is missing or stale — because a wrong settlement is exactly the kind of error a carrier can't tolerate.

ALTITUDE 4 · WHAT TO BUILD

Specify / for build.

Finally, write it down — just enough. Prioritize features, state what the system must do and how well, and stop at the line where spec becomes over-spec.

Feature prioritization

Functional requirements

Non-functional requirements

4

TOOL 4.1 · FEATURE PRIORITIZATION

Feature prioritization

You have more ideas than time. Prioritization frameworks force the trade-offs into the open so the roadmap reflects value, not volume or volume of opinions.

WHAT IT IS

Prioritization ranks what to build by weighing value against cost. Common frames: **MoSCoW** (Must / Should / Could / Won't), **RICE** (Reach × Impact × Confidence ÷ Effort), and the **value/effort matrix**. The framework matters less than the discipline: making “won't” an explicit, respected category. For a pilot, the bar is brutal — a feature earns its place only if it's needed to prove the core bet.

THE QUESTIONS IT ANSWERS

- *What must exist to **prove the bet** — and what merely would be nice?*
- *What's the **value vs. effort** of each candidate?*
- *What are you explicitly **not** building (the “won't” list)?*
- *Does the cut list match your pilot's one success metric?*

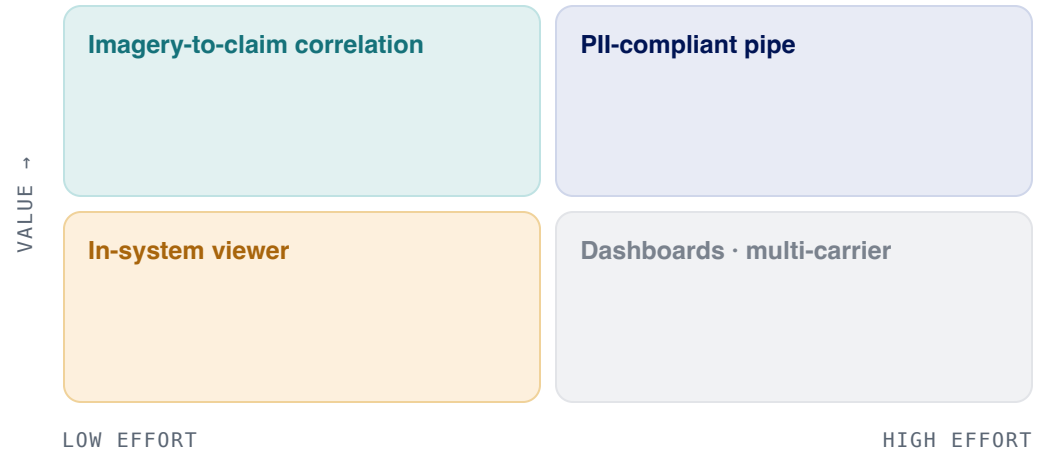
WHAT "GOOD" LOOKS LIKE

- ✓ A ranked list with explicit “won't build”
- ✓ Tied to the pilot's success metric
- ✓ Effort estimated, not just value wished for
- ✓ The list is short enough to ship in weeks

FAILURE MODE

Prioritizing by loudest voice. Without a framework, the roadmap becomes whoever argued hardest or the last customer who emailed. And gold-plating — polishing “could-haves” before “musts” work — is how pilots miss their window.

VALUE / EFFORT



AEGIS EO

Everything that didn't move **time-to-claim** went to “won't — yet.” Multi-carrier support, dashboards, extra data sources: all real, all deferred. The pilot build was only what proved the one metric.

TOOL 4.2 · FUNCTIONAL REQUIREMENTS

Functional requirements

A functional requirement states what the system must do — a specific, testable behaviour. It's the contract between the design and the build.

— WHAT IT IS

A **functional requirement** specifies a behaviour the system must exhibit: an input, an action, and an expected output, written so it can be **verified**. Good ones are atomic, unambiguous, and testable — “the system shall display the most recent imagery for a claim address within the claims view.” They flow directly from your use cases and flows; if a requirement doesn't trace back to one, question why it exists.

— THE QUESTIONS IT ANSWERS

- *What must the system **do**, stated as testable behaviours?*
- *Does each requirement trace to a **use case or flow**?*
- *Is each one **atomic and unambiguous** — one behaviour, one test?*
- *Could a builder and a tester read it the **same way**?*

— WHAT "GOOD" LOOKS LIKE

- ✓ Each requirement is testable and atomic
- ✓ Every requirement traces to a use case
- ✓ No solution-design smuggled into the “what”
- ✓ A stranger could verify each one objectively

— FAILURE MODE

Requirements that can't be tested. “The system should be intuitive” isn't a requirement — it's a wish. If you can't write the test that proves it's met, rewrite it until you can. Equally, don't over-specify the **how**; functional requirements say what, not which library.

Sample functional requirements · UC-01

FR-1	System shall ingest imagery from the carrier's existing Maxar/Planet subscription
FR-2	System shall display imagery for a claim's address inside the claims view
FR-3	System shall log time from claim-open to evidence-ready

AEGIS EO

Aegis's pilot requirements stayed tied to the one use case — each one testable against the claims workflow:

TOOL 4.3 · NON-FUNCTIONAL REQUIREMENTS

Performance / & non-functional requirements

Functional requirements say what the system does. Non-functional requirements say how well — speed, security, reliability, compliance. In regulated markets, these are often the real gatekeepers.

WHAT IT IS

Non-functional requirements (NFRs) define the **qualities** a system must have: performance, scalability, security, privacy, reliability, accessibility, compliance. They're easy to defer and dangerous to ignore — in many B2B and regulated contexts an NFR (data privacy, latency, uptime) is the actual reason a deal lives or dies. State them with numbers where you can: “under 2 seconds,” “99.9% uptime,” “no PII leaves the carrier's tenant.”

THE QUESTIONS IT ANSWERS

- *How **fast / reliable / available** must it be, in numbers?*
- *What **security and privacy** constraints are non-negotiable?*
- *Which NFR is a **gatekeeper** — a hard wall to any deal?*
- *What can you **defer** past the pilot without lying to the customer?*

PII
No personal data leaves the carrier's tenant — the gatekeeper NFR

<2s
Imagery loads inside the claims view

Audit
Every access logged for compliance review

AEGIS EO

For Aegis, the decisive requirement wasn't functional at all — it was an NFR. **PII compliance was the unspoken wall** that killed every in-house attempt. Naming it as a hard requirement on day one, and designing the pipe to satisfy the carrier's DPO, was the difference between a pilot and a dead end.

WHAT "GOOD" LOOKS LIKE

- ✓ Key NFRs stated with measurable targets
- ✓ The gatekeeper requirement identified early
- ✓ Compliance designed in, not bolted on
- ✓ Pilot-stage NFRs separated from scale-stage ones

FAILURE MODE

Treating NFRs as ‘later.’ Teams ship the features and discover at procurement that the dealbreaker was always security or privacy. The non-functional wall is invisible until you hit it — ask the gatekeeper what it is **before** you build, not after.

PUTTING IT TOGETHER

Design top-down / build bottom-up.

The toolkit reads as a descent — user, problem, solution, spec — but value flows back up: a requirement only matters if it serves a flow, which serves a use case, which serves a job a real person is trying to get done. Lose the thread at any altitude and the levels below inherit the mistake.

Altitude 1

Understand

Jobs, segments, personas, journey. Who you serve and what they're really doing.

Altitude 2

Frame

Use cases and a design sprint. Converge messy insight into a testable bet.

Altitude 3

Design

Workflows and the four user flows. How the product behaves when reality intervenes.

Altitude 4

Specify

Prioritize, then write what it must do and how well — and stop before over-spec.